

An Internship Report

Of

Virtual Internship

Submitted

To

**School of Computer Science and Engineering
IILM University, Greater Noida, U.P.**

In partial fulfilment of the requirement of degree of
B. Tech (Computer Science and Engineering)



By

Roll Number of Student: 25scs1003004922

Name of Student: Anish Kumar

Batch: 2024-2028

CANDIDATE'S DECLARATION

I, ANISH KUMAR, do hereby declare that the internship report titled “Virtual Internship” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement, and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature:-

Student Name: ANISH KUMAR

Roll No.: 25SCS1003004922

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to thank the AICTE – EduSkills Virtual Internship Program team, EduSkills Academy, and Google for Developers for providing me the opportunity and the structured platform to learn and gain hands-on exposure in the domain of Google AI-ML. I am also thankful to my institute, IILM University, Greater Noida, and the faculty of the School of Computer Science and Engineering for their continuous support and encouragement. I express my gratitude to my parents for their blessings.

Signature:-

Student Name: ANISH KUMAR

Roll No.:25SCS1003004922

TABLE OF CONTENTS

1. Candidate's Declaration	
2. Acknowledgement	ii
3. Internship Completion Certificate	
4. Project Description	
4.1 Introduction	1

3. INTERNSHIP COMPLETION CERTIFICATE



CERTIFICATE OF EXPERIENCE

To Whomsoever It May Concern

This is to certify that **Anish Kumar** successfully completed an internship with **YuvaIntern** in the role of **Junior Software Developer - Agribusiness Solution**.

Throughout the tenure of this internship, Anish demonstrated consistent dedication, professionalism, and a strong willingness to learn and grow. Their contributions were valued by the entire team at YuvaIntern, and they conducted themselves with integrity and commitment throughout the program.

We commend Anish's hard work and wish them continued success in all future endeavors.

Certificate No: YI/2026/183999/408535

Date of Issue: August 31, 2026

FOR AND ON BEHALF OF YUVAINTERN



Kounal Gupta

Founder, [YuvaIntern.com](https://yuvaintern.com)

☐ **Henry Harvin America Head Office**
Henry Harvin Inc
8 The Green, # 19614 Dover,
DE 19901, United States.

☐ **Henry Harvin Asia Pacific Office**
Henry Harvin India Education LLP
Henry Harvin House,
B-12, Sector-6, Noida(UP), India- 201301

☐ **Henry Harvin Middle East Office**
Henry Harvin Co. L.L.C.
2703, Blue Matrix, 27th floor The
Prime Tower, Business Bay, Dubai, UAE

Internship Offer Letter



Dear Anish Kumar,

We are pleased to extend to you this formal offer for an Internship position as **Junior Software Developer - Agribusiness Solution** at **YuvaIntern**. We were impressed with your background, skills, and enthusiasm, and we believe you will be a valuable addition to our team.

Internship Details:

- **Start Date:** August 03, 2026
- **End Date:** August 31, 2026
- **Duration:** 4 weeks
- **Location:** Remote

Key Responsibilities:

As a **Junior Software Developer - Agribusiness Solution** intern, you will work on tasks and projects related to As a Junior Software Developer for Agribusiness Solution, you will be responsible for developing software solutions to optimize agricultural processes and improve yield. You will work closely with agribusiness experts to understand their needs and translate them into functional software applications..

Benefits & Learning Opportunities:

Throughout the duration of your internship, you will have the opportunity to:

- Develop practical skills in [relevant skills/tools].
- Gain hands on experience in [specific tasks/projects].

As part of joining our team, you may be required to sign a confidentiality or non-disclosure agreement. You are also expected to comply with all applicable company policies and regulations, which will be provided to you upon commencement.

A handwritten signature in black ink, appearing to read "Kounal".

Kounal Gupta
Founder, YuvaIntern

4. PROJECT DESCRIPTION

1. Introduction

This report consolidates the work completed during Weeks 2 through 4 of the Software Engineering Internship Program. Over this period, the internship covered two applied software projects that together span the software development lifecycle from architectural design through implementation, testing, and post-development evaluation:

- AgriConnect — a cloud-based agribusiness platform connecting smallholder and commercial farmers with buyers, traders, and logistics providers, with IoT-based farm monitoring (Weeks 2–3).
- TaskFlow — a task management web application built on a React / Node.js (Express) / MongoDB stack, evaluated and debugged in Week 4.

The purpose of this report is to document the objectives, technical work, tools used, challenges encountered, and outcomes achieved across these weeks, and to reflect on the skills developed during the internship.

2. Internship Objectives

- Translate functional requirements into a concrete, justified software architecture for a real-world platform.
- Implement core business logic as working, testable code following sound software engineering practices.
- Apply automated unit testing and measure code quality through coverage metrics.
- Conduct structured functional, exploratory, and performance evaluation of an existing application.
- Diagnose, document, and resolve software defects using a repeatable debugging methodology.
- Produce clear, professional technical documentation suitable for stakeholder review.

3. AgriConnect: Software Architecture Design

Building on requirements gathered in Week 1 (produce listing, buyer discovery, secure payments, delivery coordination, and real-time farm monitoring), Week 2 focused on designing a layered, microservices-based architecture for AgriConnect, deployed on public cloud infrastructure.

3.1 Design Principles

- Inclusivity — core marketplace functions reachable by feature-phone farmers via USSD/SMS.
- Resilience to intermittent connectivity — offline-first mobile data capture with background sync.
- Independent scalability — marketplace, IoT telemetry, and payment workloads decoupled from one another.
- Security and trust — strong authentication, encryption, and auditability for financial and identity data.

3.2 Architecture and Diagrams Produced

- High-level system architecture (client, API Gateway, microservices, message broker, and data layers).
- Level-1 Data Flow Diagram covering Farmer, Buyer/Trader, and Payment Gateway interactions.
- Sequence diagram for the "place order" flow across the Gateway, Order, Payment, and Notification services.
- Deployment diagram mapping the logical architecture onto AWS (ECS Fargate, RDS, DocumentDB, ElastiCache, IoT Core).

3.3 Technology Stack

The stack was selected against Week 1 non-functional requirements — affordability, low-connectivity support, and independent scalability of marketplace and IoT workloads:

Layer	Technology	Key Reason
Mobile / Web Client	React Native / React.js	Shared design patterns; offline-first sync for rural connectivity
Low-connectivity Access	USSD/SMS (Africa's Talking)	Reaches feature-phone farmers without data plans
API Layer	Kong / AWS API Gateway	Centralizes auth, throttling, and routing
Backend Services	Node.js (Express) microservices	Non-blocking I/O for concurrent marketplace/IoT loads
Async Messaging	Kafka / RabbitMQ	Decouples services; buffers bursty sensor data
Relational / Document Data	PostgreSQL / MongoDB	ACID for transactions; flexible schema for sensor data
Caching, Storage, IoT	Redis, AWS S3, AWS IoT Core (MQTT)	Low-latency reads, durable file storage, lightweight device protocol
Payments	M-Pesa / Stripe	Mobile-money dominance in region plus card coverage

3.4 Security Considerations

- OAuth2/JWT authentication with role-based access control enforced at the Gateway and re-validated per service.
- TLS 1.2+ enforced on all traffic; AES-256 encryption at rest for PII and payment fields.
- Secrets managed via a dedicated secrets manager; least-privilege network segmentation in private subnets.
- Per-device X.509 certificates for IoT sensors; append-only audit logging for payment and order changes.

3.5 Outcome

The Week 2 deliverable translated the Week 1 requirements into a concrete technical design — architecture diagrams, a justified technology stack, module interface definitions, an integration strategy, and a security model — sufficient to begin sprint-level implementation planning.

4. AgriConnect: Prototype Development and Unit Testing

Week 3 moved from design to implementation, building working Node.js prototypes for two functionalities identified as critical-path items in the Week 2 architecture:

- Order placement and validation (Marketplace & Order Service) — input validation, stock checking, volume-discount pricing, and order creation.
- Sensor payload validation and normalization (IoT Sensor Ingestion Service) — schema and range checks that reject malformed or implausible readings before they enter the platform.

4.1 Engineering Approach

- Language/runtime: JavaScript (ES2020) on Node.js, matching the Week 2 backend choice for direct portability into Express services.
- Paradigm: predominantly pure functions for validation and calculation (validateOrderInput, calculateOrderTotal, validateSchema, validateRanges); side effects isolated behind a repository-style interface (getStock, getPrice, reserveStock) to enable testing against in-memory fakes.
- Error handling: custom Error subclasses (OrderValidationError, InsufficientStockError, SensorPayloadError) so callers can distinguish failure types and map them to HTTP status codes (e.g., validation → 400, stock → 409).
- Testing: Jest, chosen for zero-configuration setup, built-in assertions, and coverage tooling.

4.2 Test Results

The automated suite covered valid inputs, boundary conditions, and error scenarios for both modules:

Metric	Result
Test suites	2 passed / 2 total
Tests	30 passed / 30 total
Statement / Branch / Function / Line coverage	100% across both modules

4.3 Challenges and Resolutions

- Coverage gap on an empty/whitespace-only deviceId — closed by adding a dedicated test case, bringing branch coverage from 96.87% to 100%.
- Floating-point precision artifacts in discount calculations (e.g., 474.99999999999994 instead of 475) — resolved with a round2() helper applied at each intermediate calculation step.
- Incorrect ordering of validation vs. side effects — reordered so validation always precedes the stock check, with a regression test locking in the ordering.
- Overly generic error handling — replaced a single generic Error with distinct subclasses to support precise, testable failure handling.

4.4 Outcome

The prototype demonstrated fully unit-tested implementations of two core AgriConnect functionalities, with 100% code coverage and an error-handling model designed to carry forward directly into the full Express/MongoDB implementation.

5.TaskFlow: Evaluation, Debugging, and Performance Testing

Week 4 shifted focus to TaskFlow, a task management web application (React front end, Node.js/Express API, MongoDB data layer). The objective was to systematically validate functionality, resolve defects, and quantify performance ahead of a stakeholder pilot.

5.1 Evaluation Methodology

- Functional walkthrough of every user-facing flow (registration, login, task CRUD, filtering, sorting) against Week 2 acceptance criteria.
- Exploratory and boundary testing — empty inputs, oversized payloads, special characters, rapid repeated actions, invalid date ranges.
- Automated regression suite — Jest/Supertest (42 API test cases) plus React Testing Library for component behavior.

- Load and performance testing — Apache JMeter simulating 10, 25, and 50 concurrent virtual users against staging.

5.2 Bug Inventory

ID	Issue	Severity	Status
BUG-01	Empty-title task submission accepted	High	Resolved
BUG-02	Duplicate tasks from rapid button clicks	High	Resolved
BUG-03	Task list not updating without manual refresh	Medium	Resolved
BUG-04	API response time >2s under 50+ concurrent users	Medium	Mitigated
BUG-05	Date picker allows past due dates unwarned	Low	Resolved
BUG-06	Session tokens never expire	High	Resolved
BUG-07	Raw stack traces exposed in API error responses	Medium	Resolved
BUG-08	UI layout breaks below 375px viewport width	Low	Open / Backlog

5.3 Representative Root-Cause Fixes

- BUG-01: client-side truthy check replaced with a trimmed-length check; added a Mongoose 'required'/minlength' constraint and server-side 400 handling as a second line of defense.
- BUG-02: introduced an isSubmitting guard to disable the button during a pending request, plus a client-generated idempotency key so the server deduplicates retries within a 60-second window.
- BUG-04: MongoDB explain() revealed a full collection scan (COLLSCAN); added a compound index on {userId, createdAt} and cursor-based pagination, reducing the query plan to an IXSCAN.
- BUG-06: added an expiresIn: '2h' claim to JWT signing and a Redis-backed token blacklist checked on logout to immediately invalidate active sessions.

5.4 Performance Results

Metric	Before	After	Target
Avg. API response time	2,450 ms	680 ms	< 800 ms
Task creation success rate	91.2%	99.6%	> 99%
Duplicate task incidents / 100 submissions	7	0	0
Client-side crash rate (session)	3.1%	0.4%	< 1%
Unhandled exceptions logged / day	34	2	< 5
Mobile layout defect count	6	1	0

5.5 Recommendations Raised

- Near-term: resolve the remaining mobile layout defect (BUG-08), extend load testing beyond 50 concurrent users, add end-to-end tests (Playwright/Cypress), and introduce structured logging (Winston).
- Medium-term: real-time collaboration via WebSockets, role-based access control, due-date notifications, and a formal accessibility (WCAG 2.1 AA) audit.
- Process: adopt a code-review checklist requiring both client- and server-side validation and loading/disabled states for mutating actions; integrate load testing into the regular sprint cadence.

5.6 Outcome

Seven of eight identified defects were fully resolved and the eighth triaged into the backlog. The evaluation concluded that TaskFlow is stable enough to progress to a controlled stakeholder pilot, contingent on completing the near-term recommendations.

6. Technical Skills and Tools Acquired

Category	Skills / Tools
Architecture & Design	Microservices design, API Gateway patterns, event-driven architecture, AWS deployment modeling, data flow and sequence diagramming
Programming	JavaScript (ES2020), Node.js, functional-style validation logic, dependency-inversion via repository interfaces, custom error hierarchies
Testing & QA	Jest, Supertest, React Testing Library, unit testing, boundary/exploratory testing, code coverage analysis
Performance Engineering	Apache JMeter load testing, MongoDB query-plan analysis (explain()), indexing and pagination strategies
Security	OAuth2/JWT authentication, token expiry and revocation, TLS, encryption at rest, least-privilege network design
Documentation	Technical design documents, bug/issue reporting, stakeholder-facing evaluation reports

7. Overall Challenges and Reflections

Across the two projects, several recurring engineering lessons emerged:

- Validation must exist on both client and server — relying on a single layer (as seen in TaskFlow's BUG-01) creates a gap that automated tests alone won't catch until explicitly exercised.
- Ordering of validation before side effects is a subtle but important correctness property, worth locking in with a dedicated regression test rather than relying on code review alone.
- Performance issues often trace back to a small number of root causes (missing indexes, unbounded result sets) that are best diagnosed with the right tool for the layer — MongoDB's explain() for query plans, JMeter for load behavior.
- Precise, typed error handling (custom Error subclasses) pays off downstream by simplifying both testing and HTTP-layer error mapping.
- Specific, realistic edge cases — such as a present-but-empty deviceId — are easy to miss unless coverage tooling is used to surface untested branches.

8. Conclusion

Over Weeks 2 through 4, the internship progressed from architectural design (AgriConnect) through implementation and unit testing (AgriConnect prototype) to systematic evaluation, debugging, and performance tuning of an existing application (TaskFlow). Together, this work covered the core phases of the software development lifecycle and provided hands-on experience with microservices architecture, test-driven validation, and production-oriented debugging and performance analysis. The skills and practices developed — layered validation, typed error handling, coverage-driven testing, and load-test-informed optimization — are directly transferable to future development work and form a solid foundation for continued growth as a software engineer.